

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	Date:
Day:	Semester:

Lab 9: Imbalanced Fraud Detection

Lab Title	CLO	BT	PLO	Weightage
Imbalanced Fraud Detection F1 vs Accuracy & Cost-Sensitive Learning				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Understand challenges in imbalanced datasets
- Compare Accuracy vs F1-score
- Apply sampling techniques (up/down/SMOTE/SMOTEEN)
- Implement cost-sensitive logistic regression

Problem Statement

You are building a fraud detection system where fraudulent transactions are rare compared to normal ones.

You will analyze how class imbalance affects model performance and explore different techniques to handle imbalance.

Part A: Dataset Exploration

Task 1: Analyze Dataset 1 (60:40)

Load the provided dataset with a 60:40 class distribution (normal vs fraud). Visualize the dataset and observe how the two classes are distributed in the feature space.

```
def load_dataset_60_40():
    np.random.seed(42)

    normal = np.random.multivariate_normal(
        mean=[50, 50], cov=[[200, 80], [80, 200]], size=600
    )

    fraud = np.random.multivariate_normal(
        mean=[55, 55], cov=[[200, 80], [80, 200]], size=400 # VERY close mean
    )

    X = np.vstack([normal, fraud])
    y = np.array([0] * 600 + [1] * 400)

    return X, y
```

Task 2: Analyze Dataset 2 (90:10)

Load the provided dataset with a higher imbalance (90:10). Visualize and compare it with Dataset 1. Observe how class imbalance affects separability.

```
def load_dataset_90_10():  
    np.random.seed(42)  
  
    normal = np.random.multivariate_normal(  
        mean=[50, 50], cov=[[300, 120], [120, 300]], size=900  
    )  
  
    fraud = np.random.multivariate_normal(  
        mean=[51, 51], cov=[[300, 120], [120, 300]], size=100  
    )  
  
    X = np.vstack([normal, fraud])  
    y = np.array([0] * 900 + [1] * 100)  
  
    return X, y
```

Part B: Logistic Regression and Metrics

Task 3: Train Logistic Regression

Train a logistic regression model using cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

```
def train_logistic(X, y):  
    """  
    Train logistic regression using cross-entropy loss.  
    """  
    model = LogisticRegression(random_state=42, max_iter=1000)  
    model.fit(X, y)  
    return model
```

Task 4: Evaluate Accuracy and F1 Score

```
def evaluate_model(model, X, y):  
    """  
    Return accuracy and F1-score.  
    """  
    y_pred = model.predict(X)  
  
    accuracy = accuracy_score(y, y_pred)  
    f1 = f1_score(y, y_pred)  
  
    return accuracy, f1
```

Part C: Sampling Techniques

Task 5: Apply SMOTE

Apply: - SMOTE (synthetic minority oversampling)
Observe the effect on model performance.

```
def apply_smote(X, y):  
    """  
    Apply SMOTE.  
    """  
    smote = SMOTE(random_state=42)  
    X_resampled, y_resampled = smote.fit_resample(X, y)  
    return X_resampled, y_resampled
```

Task 6: Apply SMOTEEN

Apply: - SMOTEEN (SMOTE + cleaning)
Compare results with previous methods.

```
def apply_smoteenn(X, y):  
    """  
    Apply SMOTEENN.  
    """  
    smoteenn = SMOTEENN(random_state=42)  
    X_resampled, y_resampled = smoteenn.fit_resample(X, y)  
    return X_resampled, y_resampled
```

Part D: Cost-Sensitive Learning

Task 7: Weighted Loss Function

Modify the loss function to penalize misclassification of fraud more:

$$L = - \sum_{i=1}^N [w_1 y_i \log(p_i) + w_0 (1 - y_i) \log(1 - p_i)]$$

Experiment with different class weights and observe changes.

```
def train_weighted_logistic(X, y):  
    """  
    Train logistic regression with class weights.  
    """  
    model = LogisticRegression(  
        class_weight={0: 1, 1: 9}, random_state=42, max_iter=1000  
    )  
    model.fit(X, y)  
    return model
```

Discussion Questions

- **Why can accuracy be misleading in imbalanced datasets?**

Accuracy can create a mathematical illusion of success when dealing with imbalanced data. For example, in your 90:10 dataset, 90% of the transactions are normal and 10% are fraudulent. A standard model can simply guess Normal for every single transaction and achieve 90% accuracy without catching a single fraud case.

We saw this exactly in my code's output:

Standard Model -> Accuracy: 0.9000, F1-Score: 0.0000

The model was 90% accurate but had an F1-Score of 0, meaning it was completely useless at its actual job of detecting the minority class.

- **Which method improved F1-score the most?**

Based on the execution trace, SMOTEENN outperformed standard SMOTE (0.1805 vs 0.1664). While standard SMOTE generates synthetic data to balance the classes, it can sometimes create noisy data points that overlap with the majority class. SMOTEENN improves upon this by taking an extra "cleaning" step—it removes points from both classes that are too close to each other or misclassified by their nearest neighbors, resulting in cleaner decision boundaries and a better F1-Score. Depending on the exact weights used, the Cost-Sensitive (Weighted) Model also typically yields highly competitive F1-scores without sacrificing as much overall accuracy.

- **How does cost-sensitive learning compare to sampling methods?**

They represent two completely different paradigms for solving imbalanced data:

- **The Approach:** Sampling methods (like SMOTE) "fix the data" by synthesizing new, fake data points to artificially balance the dataset to 50:50. Cost-sensitive learning "fixes the thinking" by leaving the dataset entirely untouched (keeping it at 90:10) but changing the model's loss function to heavily penalize misses on the minority class.
- **Computational Cost:** Sampling physically expands your dataset, requiring more memory and longer training times. Cost-sensitive learning is much faster and computationally efficient because the dataset size doesn't increase.

- Risk: SMOTE carries a high risk of "data leakage" if you accidentally apply it before splitting your train and test sets. Cost-sensitive learning carries a very low risk of data leakage.

```
[4]  
0s  
--- Testing on 90:10 Dataset ---  
Standard Model -> Accuracy: 0.9000, F1-Score: 0.0000  
SMOTE Model -> Accuracy: 0.5190, F1-Score: 0.1664  
SMOTEENN Model -> Accuracy: 0.4460, F1-Score: 0.1805  
Weighted Model -> Accuracy: 0.5320, F1-Score: 0.1875
```